



GraphGPT: Graph Instruction Tuning for Large Language Models

Jiabin Tang
University of Hong Kong
Hong Kong, China
jiabintang77@gmail.com

Yuhao Yang
University of Hong Kong
Hong Kong, China
yuhao-yang@outlook.com

Wei Wei
University of Hong Kong
Hong Kong, China
weiwei1206cs@gmail.com

Lei Shi
Baidu Inc.
Beijing, China
harryshi.cs@gmail.com

Lixin Su
Baidu Inc.
Beijing, China
sulixinict@gmail.com

Suqi Cheng
Baidu Inc.
Beijing, China
chengsuqi@gmail.com

Dawei Yin
Baidu Inc.
Beijing, China
yindawei@acm.org

Chao Huang*
University of Hong Kong
Hong Kong, China
chaohuang75@gmail.com

ABSTRACT

Graph Neural Networks (GNNs) have evolved to understand graph structures through recursive exchanges and aggregations among nodes. To enhance robustness, self-supervised learning (SSL) has become a vital tool for data augmentation. Traditional methods often depend on fine-tuning with task-specific labels, limiting their effectiveness when labeled data is scarce. Our research tackles this by advancing graph model generalization in zero-shot learning environments. Inspired by the success of large language models (LLMs), we aim to create a graph-oriented LLM capable of exceptional generalization across various datasets and tasks without relying on downstream graph data. We introduce the GraphGPT framework, which integrates LLMs with graph structural knowledge through graph instruction tuning. This framework includes a text-graph grounding component to link textual and graph structures and a dual-stage instruction tuning approach with a lightweight graph-text alignment projector. These innovations allow LLMs to comprehend complex graph structures and enhance adaptability across diverse datasets and tasks. Our framework demonstrates superior generalization in both supervised and zero-shot graph learning tasks, surpassing existing benchmarks. The open-sourced model implementation of our GraphGPT is available at <https://github.com/HKUDS/GraphGPT>.

CCS CONCEPTS

• **Information systems** → **Data mining; Language models;** •
Mathematics of computing → **Graph algorithms.**

*Chao Huang is the Corresponding Author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGIR'24, July 14–18, 2024, Washington, DC, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0431-4/24/07

<https://doi.org/10.1145/3626772.3657775>

KEYWORDS

Large Language Models, Graph Learning, Instruction Tuning

1 INTRODUCTION

Graph neural networks (GNNs) have emerged as a powerful framework for analyzing and learning from graph-structured data [4, 27], enabling advancements in various domains, such as social network analysis [31, 65], recommender systems [9, 42], and biological network analysis [6, 25]. One of the key benefits of GNNs is their ability to capture the inherent structural information and dependencies present in graph data. By leveraging message passing and aggregation mechanisms, GNNs can effectively propagate and combine information across the graph, enabling them to model complex relationships and make accurate predictions.

In recent years, various GNN architectures have introduced innovations in how information is exchanged and aggregated among graph nodes. For example, graph convolutional network (GCNs) [17, 22] adapt convolutional operations to the graph domain, enabling effective feature representations. Graph attention networks (GATs) [39, 43] leverages attention mechanisms to assign different weights to neighboring nodes, allowing for more fine-grained information aggregation. Graph transformer networks (GTNs) [14, 60] incorporate self-attention and positional encoding to capture global dependencies and structural patterns in the graph. However, a notable limitation of many GNN approaches is their heavy reliance on supervised learning, which can lead to inadequate robustness and generalization when confronted with sparse and noisy data.

To enhance the generalization ability of GNNs, self-supervised learning (SSL) has emerged as a promising approach in graph representation learning. It aims to pre-train a robust graph model using auxiliary tasks on unlabeled graph data. The idea is to leverage the inherent structure and patterns within the graph itself to create meaningful self-supervisory signals. SSL-enhanced graph learning methods exhibit two primary paradigms: contrastive SSL and generative SSL. Within contrastive SSL, the emphasis lies on learning representations by contrasting positive and negative samples, with notable advancements of DGI [40] and GCA [67]. Conversely, generative SSL focuses on generating synthetic samples that closely

resemble the original graph structures with masked autoencoders, exemplified by techniques like GraphMAE [11] and S2GAE [35].

While these methods aim to generate graph embeddings that are generalizable to different downstream tasks, they often require a fine-tuning process using labels specific to the downstream graph learning scenarios. However, this reliance on labeled data from downstream tasks can restrict their generalization in practical situations where obtaining high-quality labels may not always be feasible. This limitation is particularly relevant in learning scenarios like cold-start recommendation systems or traffic flow prediction in new cities where accurate labels may be scarce or unavailable.

As a result, the objective of this research is to advance the generalization capabilities of graph models by addressing challenging and practical zero-shot learning scenarios. Inspired by the remarkable success of large language models (LLMs) in natural language processing (NLP) tasks [48], where they have demonstrated exceptional generalization abilities, this work aims to develop a graph-oriented LLM capable of achieving high generalization across diverse downstream datasets and tasks. However, effectively integrating large language models with graph learning poses non-trivial challenges.

- **C1:** Achieving a proper alignment between the structural information of a graph and the language space demands meticulous deliberation and thoughtful consideration.
- **C2:** Effectively guiding LLMs to comprehend the structural information of graphs remains a considerable challenge.
- **C3:** Endowing LLMs with the ability to reason step-by-step is important when tackling complex graph learning tasks.

To gain a deeper understanding of the limitations associated with directly prompting LLMs using purely text-based prompts for graph structure modeling, we provide illustrative examples in Figure 1. These examples facilitate a comparative analysis between our GraphGPT framework and the ChatGPT approach. We focus on a representative node classification task, where the objective is to predict the category of a given paper. In Figure 1 (a) and Figure 1 (b), we showcase the prediction results for two scenarios using ChatGPT: (1) utilizing only the input node textual data, and (2) employing text-based graph structure-aware prompts inspired by the prompt designs in recent studies [2, 5]. These figures highlight the potential limitations that arise when relying solely on text-based prompts for graph structure modeling, as evidenced by the incorrect paper node classification results presented. In contrast, our GraphGPT framework effectively addresses these limitations by preserving and leveraging the graph structural information, as shown in Figure 1 (c). It enables accurate identification of the paper category, in understanding the underlying graph structure.

Additionally, the utilization of text-based structural prompts leads to an increase in token size, which presents challenges in practical scenarios. Longer token sequences incur higher computational and memory costs, making it less feasible for real-world applications. Furthermore, existing LLMs have token limits, which further restrict the applicability of longer text-based prompts for large-scale graph structure modeling. These limitations emphasize the necessity for more efficient and scalable approaches that can effectively incorporate graph structural information into LLMs.

Contributions. To address these challenges, we propose a novel framework called GraphGPT, which aims to align Large Language

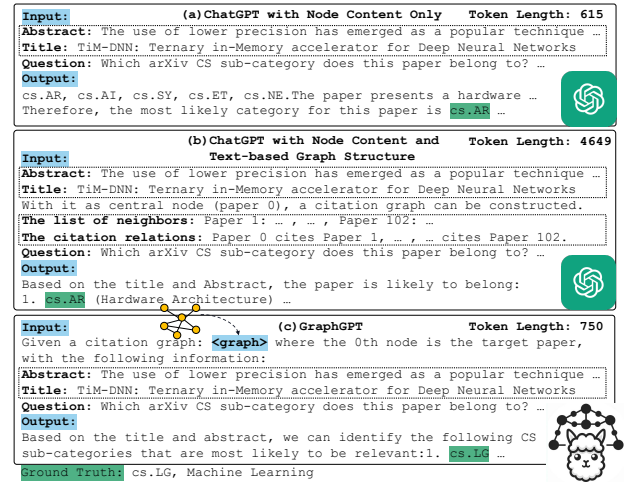


Figure 1: Limitation of LLMs in understanding graph structural contexts with heavy reliance on textual data.

Models (LLMs) with Graphs using a carefully designed graph instruction tuning paradigm. **(C1)** Our framework introduces a text-graph grounding paradigm as the initial step to align the encoding of graph structures with the natural language space. By incorporating textual information in a contrastive manner, we enable effective alignment of graph structure information within language models. **(C2)** In our proposed dual-stage graph instruction tuning paradigm, we leverage self-supervised signals through the graph matching task, which is derived from unlabeled graph structures, to serve as instructions for guiding model tuning of LLMs. By incorporating this self-supervised instruction tuning, the language model acquires domain-specific structural knowledge related to graphs, thereby enhancing its understanding of graph structures. To further customize the LLM's reasoning behavior for diverse downstream graph learning tasks, the second stage of our graph instruction tuning paradigm involves fine-tuning the LLM with task-specific graph instructions, to improve the model's adaptability. **(C3)** By incorporating the Chain-of-Thought (COT) distillation into our framework, GraphGPT enhances its step-by-step reasoning abilities and improves its performance in the face of distribution shift.

In summary, our work makes the following contributions:

- This work aims to align graph domain-specific structural knowledge with the reasoning ability of Large Language Models (LLMs) to improve the generalization of graph learning.
- Our approach aims to align LLMs with Graphs through a graph instruction tuning paradigm. This paradigm incorporates self-supervised instruction tuning, enhancing the LLM's comprehension of graph structural knowledge and its reasoning capabilities. Additionally, we introduce task-specific instruction tuning to improve the model's adaptability across diverse graph tasks.
- We evaluate our proposed GraphGPT on supervised and zero-shot graph learning tasks. We conduct thorough analyses of its component-wise effects and generalization ability. By comparing it with state-of-the-art baselines, we demonstrate the superior generalization power of our approach across various settings.

2 PRELIMINARIES

Graph-structured Data. represents information as entities (nodes) and the relationships (edges) between them. A graph is denoted as $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathbf{A}, \mathbf{X})$, comprising key components. The node set $\mathcal{V}$ represents the collection of nodes, with $|\mathcal{V}| = N$ indicating the total number of nodes. The edge set $\mathcal{E}$ characterizes the relationships between nodes. The adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ encodes the graph's topology, with each element $A_{i,j}$ indicating the presence or absence of an edge between nodes i and j . The feature matrix $\mathbf{X} \in \mathbb{R}^{N \times F}$ contains attribute or feature information associated with each node, where F represents the feature dimensionality.

Graph Neural Networks. have become a powerful framework for learning representations from graph-structured data. Unlike traditional neural networks that process grid-like data, GNNs excel in capturing the intricate relationships and dependencies within graphs. They utilize the graph's structure-comprising nodes and edges-to derive expressive node representations through repeated message propagation and aggregation operations.

$$\begin{aligned} m_v^{(l)} &= \text{Propagate}^{(l)}(\{h_u^{(l-1)} : u \in \mathcal{N}(v)\}), \\ h_v^{(l)} &= \text{Aggregate}^{(l)}(h_v^{(l-1)}, m_v^{(l)}) \end{aligned} \quad (1)$$

In Graph Neural Networks, the feature vector of node v at layer l is denoted as $h_v^{(l)}$. Message passing is performed by the $\text{Propagate}^{(l)}$ function, aggregating information from neighboring nodes of v in layer l . The $\text{Aggregate}^{(l)}$ function combines this information with the previous layer's representation of node v to update $h_v^{(l)}$. By incorporating graph structure into learned representations, GNNs can be tailored for tasks like node classification and link prediction.

3 METHODOLOGY

3.1 Structural Information Encoding with Text-Graph Grounding

To improve the understanding of graph structural information by large language models, our framework focuses on aligning the encoding of graph structures with the natural language space. This alignment enables language models to effectively comprehend the graph's structural elements using their language understanding capabilities. To achieve this, we introduce a text-graph grounding paradigm that generates prompts preserving the graph's structural context for language models. This paradigm acts as a bridge, connecting the semantic understanding of textual information with the inherent structural relationships in the graph.

In our GraphGPT, we design the graph encoder to be highly flexible, allowing it to leverage a wide range of backbone GNN architectures obtained from diverse graph pre-training paradigms. We incorporate a message-passing neural network architecture, which can be a graph transformer [60] or a graph convolutional network [17], as the structure-level pre-trained graph model. In each message-passing step, the graph encoder aggregates information from neighboring nodes, considering their relationships:

$$\mathbf{H}^{(l)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(l-1)}\mathbf{W}) \quad (2)$$

The self-loop adjacency matrix, denoted as $\tilde{\mathbf{A}}$, is obtained by adding the identity matrix $\mathbf{I}$ to the original adjacency matrix $\mathbf{A}$. $\mathbf{W}$ is the

parameter matrix. This matrix captures the self-connections and local connectivity of nodes in the graph. $\sigma(\cdot)$ is the non-linear activation. $\mathbf{H}^{(l)}$ is the graph representations at the l -th layer.

Text-Structure Alignment. To enhance the alignment of graph structure information with Language Models (LLMs), our focus is on exploring effective encoding methods that can collaborate seamlessly with LLMs. Building upon previous works [30, 49], we adopt a contrastive approach by incorporating textual information into the graph structure encoding process. We directly integrate a pre-trained graph encoder into our GraphGPT framework, enabling the seamless utilization of its capabilities. Formally, given a graph $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathbf{A}, \mathbf{X})$ with raw textual contents $\mathbf{C} = c_i \in \mathbb{R}^{l_i \times d}$, $1 \leq i \leq N$ for N nodes, we obtain encoded graph representations $\hat{\mathbf{H}} \in \mathbb{R}^{N \times d}$ and encoded text representations $\hat{\mathbf{T}} \in \mathbb{R}^{N \times d}$ as follows:

$$\mathbf{H} = f_G(\mathbf{X}), \mathbf{T} = f_T(\mathbf{C}), \hat{\mathbf{H}} = \text{norm}(\mathbf{H}), \hat{\mathbf{T}} = \text{norm}(\mathbf{T}) \quad (3)$$

We utilize the graph encoder, f_G , to generate structure-level graph representations from the input graph $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathbf{A}, \mathbf{X})$. To encode the raw textual contents $\mathbf{C}$ associated with the nodes, we employ a text encoder, such as a transformer or BERT, denoted as f_T . This step produces encoded text representations of nodes, which are then normalized row-wise using the norm function. The text-structure alignment across modalities is conducted as follows:

$$\begin{aligned} \Gamma_1 &= (\hat{\mathbf{H}}\hat{\mathbf{T}}^\top) \cdot \exp(\tau), \Gamma_2 = (\hat{\mathbf{H}}\hat{\mathbf{T}}'^\top) \cdot \exp(\tau), \Gamma_3 = (\hat{\mathbf{T}}^\top\hat{\mathbf{T}}'^\top) \cdot \exp(\tau) \\ \mathcal{L} &= \sum_{i=1}^3 \frac{1}{2} \lambda_i (\text{CE}(\Gamma_i, \mathbf{y}) + \text{CE}(\Gamma_i^\top, \mathbf{y})) \end{aligned} \quad (4)$$

where $\hat{\mathbf{T}}' = \{\frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} \hat{\mathbf{T}}_j, 1 \leq i \leq N\}$ and N is the number of nodes. In our text-graph grounding, we use the label $\mathbf{y} = (0, 1, \dots, n-1)^\top$ for the contrastive alignment objective. We employ a graph transformer [61] as the graph encoder and a vanilla transformer [38] as the text encoder.

3.2 Dual-Stage Graph Instruction Tuning

The dual-stage graph instruction tuning paradigm proposed in this work builds upon the concept of instruction tuning, which has been recently introduced to enhance the adaptability of language models for specific domains [45]. In this paradigm, we aim to align the language capacity of the model with the nuances of graph learning tasks, enabling the language model to generate more accurate and contextually appropriate responses for graph-structured data.

3.2.1 Self-Supervised Instruction Tuning. In the initial stage of our graph instruction tuning, we introduce self-supervised instruction tuning. This mechanism enhances the language model's reasoning abilities by incorporating graph domain-specific structural knowledge and effectively understanding contextual information within the graph's structure. To achieve this, we utilize self-supervised signals derived from unlabeled graph structures as instructions for model tuning. Specifically, we design a structure-aware graph matching task that guides the language model in differentiating between graph tokens using language tokens. This instruction task plays a vital role in accurately associating graph tokens with their corresponding textual descriptions, deepening the model's comprehension of the graph with the provided guidance.

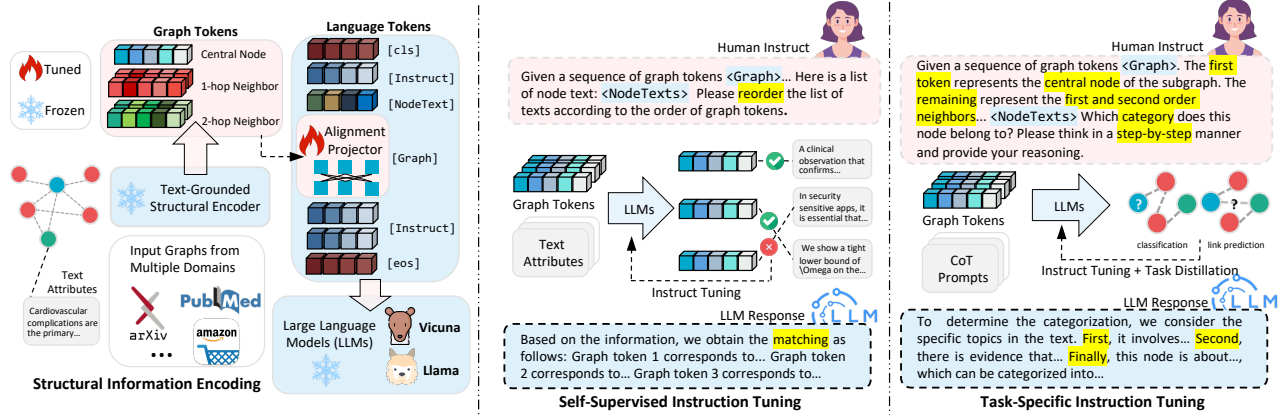


Figure 2: The overall architecture of our proposed GraphGPT with graph instruction tuning paradigm.

Instruction Design. The instruction for our graph matching task consists of three components: i) graph information, ii) human question, and iii) GraphGPT response. In this task, we treat each node in the graph as a central node and perform h-hops with random neighbor sampling, resulting in a subgraph structure. The natural language input for the LLM is the human question. In the context of the graph matching task, the instruction includes the indicator token `<graph>` and a shuffled list of node text information. For example, in a citation graph, the node text information corresponds to paper titles. The objective of the LLM in the graph matching task is to align each graph token with its corresponding node text information. This requires reordering the node text information list based on the sequence of graph tokens, effectively associating each graph token with its relevant textual description. The detailed designs of graph matching are shown in Figure 4.

Tuning Strategy. To optimize the tuning process efficiently, we propose incorporating a **Lightweight Alignment Projector**. During training, we focus on optimizing the parameters of the projector f_p , while keeping the parameters of both the LLM and the graph encoder fixed. We assume that the projector successfully learns to map the encoded graph representation to graph tokens, while the LLM excels at aligning these tokens with diverse node text information. To align the graph tokens with the language tokens, we employ a projector f_p , which can be as simple as a single linear layer. This projector establishes correspondence between the graph tokens and the language tokens. By replacing the indicator token `<graph>` in the original language token sequence, the aligned graph tokens create a modified token sequence for the LLM. This modified sequence, denoted as $\{<graph_begin>, <graph_token>_1, \dots, <graph_token>_n, <graph_end>\}$, corresponds to the number of nodes n in the graph associated with the given prompt. Given that the graph matching process is unsupervised, we have the opportunity to leverage a vast amount of unlabeled graph data from different domains, to enhance the generalizability of the learned projector. Mathematically, with projected graph tokens $X_G = f_p(\hat{H})$ and text embeddings $X_I = \text{tokenizer}(\text{instruction})$, for a sequence of length L , we compute the probability of generating the target output X_O as follows:

$$p(X_O|X_G, X_I) = \prod_{i=1}^L p_\theta(x_i|X_G, X_{I,<i>, X_{O,<i>}) \quad (5)$$

where θ are the learnable parameters within GraphGPT.

3.2.2 Task-Specific Instruction Tuning. In the second stage, we introduce task-specific instruction tuning to customize the model’s reasoning behavior for different graph learning tasks, such as node classification or link prediction. By fine-tuning the LLM using task-specific graph instructions, we guide the model to generate responses that align with the constraints and requirements of the specific graph learning task. This enhances the model’s adaptability and performance in handling diverse graph learning tasks.

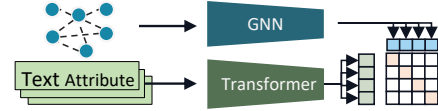


Figure 3: Workflow of text-structure alignment.

Instruction Design. We utilize a consistent instruction template comprising three parts. To generate graph information for each node, we employ the same neighbor sampling approach as in the first stage. This approach ensures the inclusion of relevant graph information, with each node serving as the central node. For the node classification task, the human question instruction includes the indicator token `<graph>` and specific text information about the central node. This instruction guides the language model to predict the category of the central node based on both the graph structure data and the accompanying text information. Figure 4 provides instruction examples for different tasks, visually illustrating the presentation of the instruction to the language model.

Tuning Strategy. In the second stage of training, we utilize the parameters of the structure-aware projector that were trained in the first stage as the initial state. This allows us to conduct instruction tuning specifically for downstream tasks. During this training process, we keep the parameters of the language model (LLM) and graph encoder fixed, focusing solely on optimizing the parameters of the projector from the previous stage. By doing so, we ensure that the LLM further aligns with the requirements of downstream tasks, enhancing its ability to comprehend and interpret graph structures.

After completing the two training stages as described above, we have confidence that our GraphGPT has acquired the capability to comprehend the given graph structure and perform downstream

Graph Information: <graph>: Central Node: 68442, Edge index: [[...src node...], [...dst node...]], Node list: [...]		Graph Matching
Human Question: Given a sequence of graph tokens <graph> that constitute a subgraph of a citation graph, ... Here is a list of paper titles: 1. ... 2. ..., please reorder the list of papers according to the order of graph tokens.		
GraphGPT Response: Based on the given graph tokens and the list of paper titles, we obtain the matching of graph tokens and papers: Graph token 1 corresponds to smt based induction methods for timed systems. Graph token 2 corresponds to ...		
Graph Information: <graph>: Central Node: 2, Edge index: [[...src node...], [...dst node...]], Node list: [...]		Node Classification
Human Question: Given a citation graph: <graph> where the 0th node is the target paper, with the following information: Abstract: ... Title: ... Question: Which arXiv CS sub-category does this paper belong to? ...		
GraphGPT Response: cs.IT, cs.LG, cs.SP, cs.CV, cs.NA. The paper discusses the Restricted Isometry ... So, it is likely to belong to cs.IT...		
Graph Information: <graph>: Central Node 1: 8471, Edge index 1: [[...src node...], [...dst node...]], Node list 1: [...]		Link Prediction
Graph Information: <graph>: Central Node 2: 19368, Edge index 2: [[...src node...], [...dst node...]], Node list 2: [...]		
Human Question: Given a sequence of graph tokens: <graph> that constitute a subgraph of a citation graph, ... Abstract: ... Title: ... and the other sequence of graph tokens: <graph>, ... Abstract: ... Title: ..., are these two central nodes connected? Give me an answer of "yes" or "no".		
GraphGPT Response: Yes, they are connected. Based on the first paper, ... And the second paper proposes ...		

Figure 4: Our instruction designs for graph matching task (upper), node classification (middle) and link prediction (lower).

tasks on the provided graph. The training process involving instruction tuning and the freezing of specific model parameters has refined the model’s understanding of graph structures, enabling it to effectively tackle various tasks associated with the given graph.

3.3 Chain-of-Thought (CoT) Distillation

When faced with diverse graph data, language models may encounter unfamiliar patterns and structures, leading to challenges in generating accurate and coherent responses. This is especially true when the number of node classes varies across different types of graph data, causing distribution shift. To address this challenge and enhance accuracy in the presence of distribution shift, it is crucial to equip our GraphGPT with step-by-step reasoning abilities. Thus, we propose incorporating the Chain-of-Thought (COT) technique [47], which explicitly models the flow of thoughts and reasoning steps. By leveraging COT, our language model improves the coherence and consistency of generated text, enabling it to follow a logical progression of ideas and enhance its understanding and reasoning capabilities for the given graph data.

Incorporating the Chain-of-Thought (COT) technique can be challenging due to the influence of model parameter scale [32]. To overcome this, we draw inspiration from previous research [32] and adopt a distillation approach. By extracting valuable knowledge from a closed-source, powerful language model like ChatGPT (with over 200 billion parameters), we can generate high-quality COT instructions and enhance our model’s COT reasoning capabilities without increasing the parameter count.

COT Distillation Paradigm. Our approach involves designing tailored Chain-of-Thought (COT) prompts for node-specific tasks. For the node classification task in a citation graph, we provide the abstract, paper title, and a task description as input. Using the GPT-3.5 language model (LLM), we incorporate "Please think about the categorization in a step-by-step manner." to enable step-by-step reasoning. By engaging in sequential thought, the LLM generates output that includes predictions for node classes and detailed explanations for each prediction. This ensures transparent and comprehensible reasoning and decision-making. To further enhance performance, we integrate the generated COT instruction data with previously designed instructions for task-specific instruction tuning. With the integrated instructions, we proceed with the proposed instruction tuning paradigm.

4 EVALUATION

We conduct experiments to address key research questions:

- **RQ1:** How does the proposed GraphGPT framework perform in both supervised and zero-shot graph learning settings?
- **RQ2:** What is the generalization ability of our model in handling multiple tasks without experiencing catastrophic forgetting?
- **RQ3:** What is the contribution of various key components in the proposed GraphGPT framework to its overall performance?
- **RQ4:** How scalable and efficient is our GraphGPT framework?

4.1 Experimental Settings

4.1.1 Data Descriptions. We evaluate our GraphGPT using three datasets: OGB-arxiv, PubMed, and Cora. The OGB-arxiv dataset [12] represents a directed graph capturing the citation network among computer science arXiv papers indexed by MAG [41]. Each paper is manually labeled with a research category selected from 40 subject areas. The PubMed dataset [8] consists of 19,717 scientific publications on diabetes from the PubMed database, categorized into Experimental induced diabetes, Type 1 diabetes, and Type 2 diabetes. Additionally, it includes a citation network with 44,338 links. The Cora dataset [49] comprises 25,120 research papers connected through citations. We use an expanded version with 70 classes, larger than previous versions [17].

4.1.2 Evaluation Protocols. To facilitate comparison across different datasets, we map node features into a unified vector space by encoding raw text information with a pre-trained BERT [3]. In our experiments, we partition the Cora and PubMed datasets into training, validation, and testing sets following a 3:1:1 ratio, as described in previous works [8, 49]. For the OGB-arxiv data, we adhere to the public split setting [12] with a training, validation, and testing ratio of 6:2:3. To evaluate our model’s performance, we utilize three commonly used metrics: Accuracy and Macro F1 for node classification, and AUC for link prediction.

4.1.3 Baseline Methods. In our performance comparison, we consider various state-of-the-art methods for comprehensive evaluation. (i) The first category includes MLP, which employs a Multi-layer Perceptron for node representation. (ii) The second category comprises representative graph neural encoders, including GraphSAGE [7], GCN [17], GAT [39], and RevGNN [21]. (iii) The third category focuses on the self-supervised approach DGI [40] for graph learning. (iv) The fourth category explores knowledge distillation-enhanced GNNs, with GKD [55] and GLNN [63] as notable methods. (v) The fifth category showcases recently proposed strong graph transformer networks, with NodeFormer [51] and DIFFormer [50] as competitors. (vi) Lastly, we consider open-sourced LLMs, such

Table 1: Performance comparison of various methods on node classification under both supervised and zero-shot settings.

Dataset	Arxiv-Arxiv		Arxiv-PubMed		Arxiv-Cora		(Arxiv+PubMed)-Cora		(Arxiv+PubMed)-Arxiv	
Model	Accuracy	Macro-F1	acc	Macro-F1	Accuracy	Macro-F1	Accuracy	Macro-F1	Accuracy	Macro-F1
MLP	0.5179	0.2536	0.3940	0.1885	0.0258	0.0037	0.0220	0.0006	0.2127	0.0145
GraphSAGE	0.5480	0.3290	0.3950	0.1939	0.0328	0.0132	0.0132	0.0029	0.1281	0.0129
GCN	0.5267	0.3202	0.3940	0.1884	0.0214	0.0088	0.0187	0.0032	0.0122	0.0008
GAT	0.5332	0.3118	0.3940	0.1884	0.0167	0.0110	0.0161	0.0057	0.1707	0.0285
RevGNN	0.5474	0.3240	0.4440	0.3046	0.0272	0.0101	0.0217	0.0016	0.1309	0.0126
DGI	0.5059	0.2787	0.3991	0.1905	0.0205	0.0011	0.0205	0.0011	0.5059	0.2787
GKD	0.5570	0.1595	0.3645	0.2561	0.0470	0.0093	0.0406	0.0037	0.2089	0.0179
GLNN	0.6088	0.3757	0.4298	0.3182	0.0267	0.0115	0.0182	0.0092	0.3373	0.1115
NodeFormer	0.5922	0.3328	0.2064	0.1678	0.0152	0.0065	0.0144	0.0053	0.2713	0.0855
DIFFormer	0.5986	0.3355	0.2959	0.2503	0.0161	0.0094	0.0100	0.0007	0.1637	0.0234
baichuan-7B	0.0946	0.0363	0.4642	0.3876	0.0405	0.0469	0.0405	0.0469	0.0946	0.0363
vicuna-7B-v1.1	0.2657	0.1375	0.5251	0.4831	0.1090	0.0970	0.1090	0.0970	0.2657	0.1375
vicuna-7B-v1.5	0.4962	0.1853	0.6351	0.5231	0.1489	0.1213	0.1489	0.1213	0.4962	0.1853
GraphGPT-7B-v1.1-cot	0.4913	0.1728	0.6103	0.5982	0.1145	0.1016	0.1250	0.0962	0.4853	0.2102
GraphGPT-7B-v1.5-stage2	0.7511	0.5600	0.6484	0.5634	0.0813	0.0713	0.0934	0.0978	0.6278	0.2538
GraphGPT-7B-v1.5-std	0.6258	0.2622	0.7011	0.6491	0.1256	0.0819	0.1501	0.0936	0.6390	0.2652
GraphGPT-7B-v1.5-cot	0.5759	0.2276	0.5213	0.4816	0.1813	0.1272	0.1647	0.1326	0.6476	0.2854
p-val	$2.26e^{-9}$	$1.56e^{-10}$	$2.22e^{-7}$	$1.55e^{-9}$	$1.04e^{-9}$	$9.96e^{-6}$	$7.62e^{-8}$	$1.97e^{-7}$	$1.5e^{-13}$	$4.63e^{-6}$

as Baichuan-7B, vicuna-7B-v1.1, and vicuna-7B-v1.5 as baselines for understanding text-attributed graph data.

4.1.4 Implementation Details. For our model implementation, we primarily use the PyTorch and Transformers libraries. We utilize Vicuna-7B-v1.1 and Vicuna-7B-v1.5 as the base models. The batch size is set to 2 per GPU, and the learning rate is $2e^{-3}$. We apply a warmup ratio of $3e^{-2}$ and set the maximum input length of the Large Language Model (LLM) to 2048. The training process runs for 3 epochs. In the task-specific instruction tuning stage, we explore various combinations of instruction data to assess the model's performance under different data mixtures. The hyperparameter settings remain constant, except for the number of training epochs, which is set to 2 in this stage. The alignment projector parameters fine-tuned in the self-supervised instruction tuning stage serve as the initial parameters for the projector in the second tuning stage. For evaluating most baselines, we use their publicly available code. For more implementation details, please refer to our released code.

4.2 Overall Performance Comparison (RQ1)

We conduct experiments on the node classification task, evaluating both supervised and zero-shot scenarios. The overall performance is summarized in Table 1. In the **Supervised Task Setting**, models are trained on a specific dataset and evaluated on the corresponding test set (e.g., training on Arxiv-Arxiv and testing on the Arxiv test set). In the **Zero-Shot Task Setting**, models are trained on a specific dataset and tested on other datasets without additional training (e.g., training on Arxiv-PubMed and testing on the PubMed dataset). To handle variations in the number of classes across datasets, we employ a transfer-trained classifier, such as a linear layer, when testing GNN-based models. In Table 1, "-7B-" indicates the parameter scale, while "-v1.1-" and "-v1.5-" represent different versions of the base Vicuna model. "-stage2" indicates the adoption of only the second stage tuning. "-std" and "-cot" denote the use of the standard and generated COT instruction datasets, respectively.

Obs.1: Overall Superiority of our GraphGPT. Our graph LLM consistently outperforms various state-of-the-art baselines in both

supervised and zero-shot scenarios. Notably, even recently developed strong GNN-based models, such as NodeFormer, DIFFormer, and GKD, exhibit good structural modeling capabilities in the supervised setting. However, when transferred to new datasets without further training, their performance significantly declines. In contrast, our GraphGPT not only surpasses all state-of-the-art methods in supervised tasks but also achieves a remarkable 2-10 times increase in accuracy in the zero-shot graph learning scenario.

LLM-based solutions like Baichuan-7B and Vicuna-7B maintain stable performance across different datasets but rely solely on text information for predictions. In contrast, our GraphGPT preserves graph structure, providing a comprehensive solution for graph learning tasks. Two key factors contribute to these improvements: (i) Our dual-stage graph instruction tuning aligns structural information encoded by the graph encoder with natural language tokens, enabling the LLM to understand the graph's inherent characteristics. (ii) Our framework facilitates mutual enhancement between the graph encoder and LLM, filling the LLM's gap in structural understanding and enabling it to reason about the graph's structure.

Obs.2: Benefits with Structure-aware Graph Matching. The presence of the first stage, which involves self-supervised graph matching tasks for instruction tuning, plays a crucial role in enhancing the zero-shot transferability of our GraphGPT. The first stage focuses on aligning the graph tokens, which encode rich structural information, with the language tokens. This alignment enables the model to develop a deeper understanding of the inherent structural characteristics of the graph data. Without the first stage, if we only conduct the second stage of task-specific instruction tuning, the model tends to be more prone to overfitting on the specific dataset. In such cases, the model's performance may be heavily reliant on dataset-specific patterns and characteristics, rather than a genuine understanding of the underlying graph structure. This can limit the model's ability to generalize to new, unseen datasets.

Obs.3: Benefits with COT Distillation. The "-std" and "-cot" variants indicate that the use of COT distillation substantially benefits more complex graph learning tasks. Models tuned with the standard

Table 2: Performance comparison of various instruction mixtures in supervised learning on the Arxiv dataset and the zero-shot setting on the Cora dataset for node classification.

Dataset	Supervision. on Arxiv		Zero Shot on Cora	
Model	Acc	Macro-F1	Acc	Macro-F1
MLP	0.5179	0.2536	0.0220	0.0006
GraphSAGE	0.5480	0.3290	0.0132	0.0029
GCN	0.5267	0.3202	0.0187	0.0032
GAT	0.5332	0.3118	0.0161	0.0057
RvGNN	0.5474	0.3240	0.0217	0.0016
DGI	0.5059	0.2787	0.0205	0.0011
GKD	0.5570	0.1595	0.0406	0.0037
GLNN	0.6088	0.3757	0.0182	0.0092
NodeFormer	0.5922	0.3328	0.0144	0.0053
DIFFormer	0.5986	0.3355	0.0100	0.0007
baichuan-7b	0.0946	0.0363	0.0405	0.0469
vicuna-7B-v1.1	0.2657	0.1375	0.1090	0.0970
vicuna-7B-v1.5	0.4962	0.1853	0.1489	0.1213
Arxiv-std + PubMed-std	0.6390	0.2652	0.1501	0.0936
Arxiv-cot + PubMed-cot	0.6476	0.2854	0.1647	0.1326
Arxiv-mix + PubMed-mix	0.6139	0.2772	0.1544	0.1048
Arxiv-std + PubMed-std + Link	0.5931	0.2238	0.1847	0.1579
Arxiv-mix + PubMed-mix + Link	0.6874	0.3761	0.1836	0.1494

Table 3: Performance comparison of various instruction mixtures for link prediction on PubMed.

Dataset	PubMed	
Model	AUC	AP
MLP	0.5583	0.5833
GAT	0.5606	0.6373
GraphSAGE	0.5041	0.5813
RevGNN	0.4538	0.5083
Node2Vec	0.6535	0.6885
w/o Link	0.5010	0.5005
only Link	0.6704	0.6087
Arxiv-std + PubMed-std + Link	0.8246	0.8026
Arxiv-mix + PubMed-mix + Link	0.6451	0.5886

instruction dataset can already achieve prominent results when transferred to simpler tasks, such as the PubMed dataset with 3 classes, with an accuracy of 0.7011 for Arxiv-PubMed. However, their performance tends to be mediocre when applied to complex tasks like the Cora dataset with 70 classes. By leveraging the powerful reasoning capabilities of the closed-source model (GPT-3.5) through COT distillation, our model can integrate this knowledge and significantly enhance its performance on complex graph tasks.

4.3 Generalization Ability Investigation (RQ2)

In this subsection, we explore the generalization ability of our model by incorporating more instruction data to fine-tune the LLM for effectively handling various types of tasks. Our main results and experimental observations are presented as follows:

More Data Boost Model Transfer Ability. In our preliminary investigation, we examine the influence of data quantity on the transfer capability of our GraphGPT, as illustrated in the "(Arxiv + PubMed)-Cora" column of Table 1. In this experiment, we train models using a combination of the Arxiv and PubMed datasets and perform zero-shot testing on the Cora dataset. The results reveal that by incorporating a relatively smaller PubMed dataset (with 20,000+ items) alongside Arxiv, our GraphGPT exhibits a significant improvement in transfer performance on Cora. In contrast, the transfer performance of GNN-based models, trained separately on Arxiv and PubMed, actually deteriorates.

More Data Yet No Forgetting. We further validate the performance of the combined Arxiv and PubMed instruction data on the

Table 4: Module ablation study under both supervised and zero-shot settings to analyze the individual contributions.

Dataset	Arxiv-Arxiv		Arxiv-PubMed		Arxiv-Cora	
Variant	Acc	Mac-F1	Acc	Mac-F1	Acc	Mac-F1
w/o GS	0.4962	0.1853	0.6351	0.5231	0.1489	0.1213
w/o LR	0.5807	0.2462	0.2523	0.1925	0.0050	0.0016
ours	0.6258	0.2622	0.7011	0.6491	0.1813	0.1272

original Arxiv data, as demonstrated in the "(Arxiv + PubMed)-Arxiv" column in Table 1. The results indicate that most traditional GNN-based approaches experience a significant decline in performance on Arxiv after iterative training. In contrast, our model exhibits improved performance. We attribute this phenomenon to the occurrence of catastrophic forgetting in GNN-based models, where the structural modeling competence of the model trained solely on the smaller PubMed dataset is compromised. However, our model effectively mitigates this issue through our unified graph instruction tuning paradigm. This enables our model to maintain and even enhance its performance by retaining the generalized graph structure patterns despite incorporating additional data.

Generalization for Multitasking Graph Learner. Recent studies on instruction tuning suggest that mixing different instruction tuning data can further enhance the performance of Language and Logic Models (LLMs). In this study, we ensure a consistent number of instruction entries and mix different types of instruction data, including standard instruction ("-std"), COT instruction ("-cot"), a blend of standard (50%) and COT (50%) instruction ("-mix"), and link prediction instruction ("Link"). The results are presented in Tables 2 and Table 3. We can observe that effective data mixture solutions can significantly improve the performance of our GraphGPT under various settings. The addition of task-specific instruction for link prediction task notably enhances the performance of our model in node classification. Interestingly, after incorporating node classification, the performance of link prediction also exceeds that of the selected best-performing existing models. After mixing the instructions of different tasks, our model demonstrates the ability to effectively handle various graph learning tasks and transfer its knowledge to other unseen datasets.

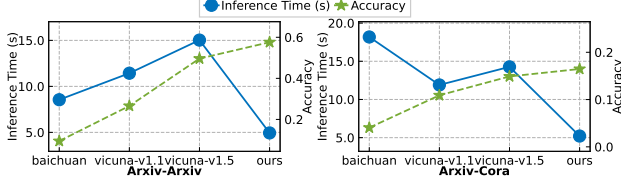
4.4 Module Ablation Study (RQ3)

We conduct an ablation study to investigate the individual contributions of different sub-modules of our proposed framework, and the results are reported in Table 5. The observations are as follows:

Effect of Graph Instruction Tuning. In our study, we investigate the benefit of incorporating graph structural information into LLM using the variant "w/o GS." In this variant, we directly adopt the base LLM (specifically, Vicuna-7B-v1.5) to perform node classification on three datasets, without incorporating graph structural information. The results of our study demonstrate that our model significantly outperforms the base model that lacks structural information. This indicates that our graph instruction tuning paradigm enables the LLM to understand the graph structural information more effectively. Importantly, this improvement in performance was achieved without altering the original parameters of the LLM. Instead, it was solely accomplished through our lightweight alignment projector, which aligns graph tokens and natural language

Table 5: Study on the time and space efficiency of our GraphGPT during both the training and inference stages.

Variants	Training Time	Tuned Parameters	GPU Occupy
Stage-1-tune	OOM	6,607,884,288	OOM
Stage-1-freeze	22:53:33	131,612,672	39517.75
improvement	-	$\downarrow \times 50.21$	-
Stage-2-tune	OOM	6,607,884,288	OOM
Stage-2-freeze	03:44:35	131,612,672	38961.75
improvement	-	$\downarrow \times 50.21$	-

**Figure 5: Inference efficiency study of our GraphGPT.**

tokens through the 1-linear projection operation.

Effect of LLM-enhanced Semantic Reasoning. We conduct further investigations to assess the influence of the LLM’s reasoning ability in our GraphGPT by performing supervised and zero-shot predictions using only the default graph encoders. This variant is denoted as "w/o LR". The results of our study indicate that our GraphGPT, which integrates the LLM, significantly enhances the performance of graph encoder, especially in the zero-shot setting. This suggests that the rich semantic information injected by the LLM provides a substantial gain in performance.

4.5 Model Efficiency Study (RQ4)

The study aims to assess the computational efficiency of our model during both the model training and inference stages.

Training Efficiency with Graph Instruction Tuning. Our instruction tuning framework follows a two-stage process where the parameters of both the LLM and the graph encoder are frozen, and only the graph-text alignment projector is tuned. We conduct a comparison between freezing and tuning the LLM parameters in a 4-card 40G Nvidia A100 environment, denoted by "-freeze" and "-tune" respectively. The study analyze the time and space efficiency in terms of training time, the number of tuned parameters, and GPU occupancy (MiB per GPU). Under the same experimental conditions, when tuning LLM parameters, we encounter Out of Memory (OOM) errors even with a batch size of 1. However, by utilizing our tuning strategy, the training process remains stable even with a batch size of 2. Moreover, the number of tuned parameters decreases by more than 50 times compared to the freezing stage.

Model Inference Efficiency. In our exploration, we assess the inference speed and accuracy of our GraphGPT by comparing it with baichuan-7B, vicuna-7B-v1.1, and vicuna-7B-v1.5 LLMs. Using a single 40G Nvidia A100, we measure inference time (seconds per response) on the Arxiv and Cora COT instruction datasets, as shown in Figure 5. Our graph LLM demonstrates superior efficiency and accuracy. Lower inference time doesn’t necessarily mean better performance: baichuan-7B yields quick but often incorrect or irrelevant answers, while vicuna-7B-v1.1 and vicuna-7B-v1.5 require longer, complex reasoning steps for better answers. In contrast, our model achieves accurate predictions through a brief reasoning process, enhancing inference efficiency.

4.6 Model Case Study (RQ5)

We conduct a detailed analysis of our model’s performance in downstream graph learning tasks compared to traditional LLMs using different types of instructions. We evaluate ChatGPT and our GraphGPT using Arxiv data, with prompts based on node content, node content with text-based graph structure, and our designed graph instruction. The results, shown in Table 6, clearly demonstrate that despite its massive parameter count (over 200B), ChatGPT struggles to make accurate predictions solely based on node text information or node content with text-based graph structure. This challenge is particularly evident when dealing with papers that have cross-disciplinary characteristics, as seen in the example of research domains in machine learning and hardware architecture. In contrast, our GraphGPT consistently provides accurate predictions and reasonable explanations. This is because our GraphGPT incorporates a subgraph structure with 103 nodes, allowing it to extract rich structural information from neighboring nodes’ citation relationships, leading to accurate predictions.

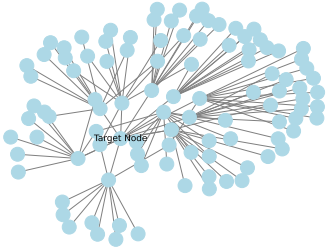
Furthermore, we believe that our approach of using graph tokens to represent the graph structure as input to the LLM is more efficient than the natural language solution. In the case of a 103-node subgraph, our GraphGPT only requires 750 tokens to be fed into the LLM, while the text-based method requires 4649 tokens. This significant reduction in token consumption translates to a substantial decrease in training and inference resource requirements.

5 RELATED WORK

Self-supervised Learning and Pre-training on Graphs. To enhance the robustness of graph models, self-supervised learning (SSL) has been introduced as a powerful technique [13, 16, 24]. It allows GNNs to learn meaningful graph representations without heavily relying on labeled data. The core idea behind self-supervised learning in graph models is to design pretext tasks that leverage the graph’s intrinsic properties to generate additional supervision signals [52]. SSL-enhanced graph learning methods can be broadly classified into two main paradigms: contrastive SSL and generative SSL. In particular, i) **Contrastive SSL** focuses on learning representations by contrasting positive and negative samples. Notable methods in this domain include GraphCL [59] and GCA [67]. Recent advancements in contrastive SSL include automated contrastive augmentation (*i.e.*, JOAO [58], AdaGCL [15]), Hyperbolic-Euclidean dual space contrasting (*e.g.*, DSGC [56]), or community-aware contrastive learning (*e.g.*, gCooL [20]). ii) **Generative SSL**, on the other hand, focuses on generating realistic samples that resemble the original graph structures. Recent advancements in this line include GraphMAE [10, 11] for feature masking, and S2GAE [35], AutoCF [53] for reconstructing masked edges as SSL tasks.

Prompt-Tuning for Graph Neural Networks. Recent efforts in enhancing the generalization capability of graph neural networks (GNNs) have focused on training GNN models in a self-supervised manner, followed by fine-tuning on specific downstream tasks using prompt-tuning techniques [64]. For example, GPPT [33] is a transfer learning paradigm, where GNNs are first pre-trained on masked edge prediction and then prompted with token pairs for downstream node classification. GraphPrompt [26] aims to handle downstream tasks by integrating pre-training and downstream tasks

Table 6: Comparison of prediction results between our GraphGPT and ChatGPT.

Title: TiM-DNN: Ternary in-Memory accelerator for Deep Neural Networks. (with 102 first-order and second-order neighbors)	
Ground-Truth Category: cs.LG, Machine Learning	
ChatGPT (Node Content Only): cs.AR, cs.AI, cs.SY, cs.ET, cs.NE. The paper presents a hardware accelerator for deep neural networks (DNNs) that is specifically designed to execute ternary DNNs, which represent weights and activations by signed ternary values. Therefore, the most likely category for this paper is cs.AR (Hardware Architecture), which focuses on the design and analysis of hardware and systems for AI and machine learning applications. The paper also involves the optimization of compute and storage requirements of DNNs, which falls under the category of cs.AI (Artificial Intelligence). Additionally, the paper proposes a programmable, in-memory accelerator, which is a type of embedded system, and therefore, it could be categorized under cs.SY (Systems and Control). Furthermore, the paper discusses the use of ternary values for DNNs, which is an emerging technique to optimize the compute and storage requirements of DNNs, and therefore, it could be categorized under cs.ET (Emerging Technologies). Lastly, the paper proposes a new bit-cell called Ternary Processing Cells (TPCs), which could be categorized under cs.NE (Neural and Evolutionary Computing).	
ChatGPT (Node Content and Text-based Graph Structure): Based on the title "TiM-DNN: Ternary In-Memory Accelerator for Deep Neural Networks", the paper is likely to belong to the following arXiv CS sub-categories: 1. cs.AR (Hardware Architecture): The paper proposes a hardware accelerator, TiM-DNN, specifically designed for executing ternary deep neural networks. This falls under the domain of hardware architecture research. 2. cs.NE (Neural and Evolutionary Computing): The paper focuses on accelerating deep neural networks, which is a topic within the field of neural and evolutionary computing. 3. cs.CV (Computer Vision and Pattern Recognition): Deep neural networks are widely used in computer vision tasks, and the TiM-DNN accelerator could potentially be applied to accelerate computer vision algorithms. 4. cs.SY (Systems and Control): The TiM-DNN accelerator is a system-level solution for accelerating deep neural networks, so it could be categorized under systems and control research. 5. cs.DC (Distributed, Parallel, and Cluster Computing): While the paper does not explicitly mention distributed or parallel computing, the TiM-DNN accelerator is designed to optimize the compute and storage requirements of deep neural networks, which are often executed on distributed or parallel computing systems.	
GraphGPT: Based on the title and abstract of the paper, we can identify the following arXiv CS sub-categories that are most likely to be relevant:	
 1. cs.LG (Machine Learning): This sub-category is relevant because the paper discusses the use of ternary DNNs for language modeling and image classification, which are both tasks that involve learning. 2. cs.AI (Artificial Intelligence): This sub-category is relevant because the paper proposes a new accelerator for DNNs, which is designed to optimize the compute and storage requirements of complex DNNs. 3. cs.NE (Numerical Analysis): This sub-category is relevant because the paper discusses the use of ternary DNNs, which involve signed ternary values, and the use of TiM tiles, which are specialized memory arrays that perform massively parallel signed vector-matrix multiplications on ternary values. 4. cs.SY (Systems and Control Theory): This sub-category is relevant because the paper discusses the implementation of TiM-DNN in 32nm technology using an architectural simulator calibrated with SPICE simulations and RTL synthesis. 5. cs.AR (Hardware Architecture): This sub-category is relevant because the paper proposes a new accelerator for DNNs, which is designed to be programmable and in-memory.	

into a unified task template. Additionally, Sun *et al.* [34] presents a unified prompt format, reformulates tasks to the graph level, and incorporates meta-learning techniques to improve multi-task performance in graph prompting. Despite these advances, these methods still require further fine-tuning that relies on supervision labels from downstream tasks to ensure accurate learning. In contrast, this work addresses this limitation by introducing a foundational graph model that tackles the more challenging task of zero-shot graph learning. By eliminating the need for label inputs from downstream tasks, this approach allows for a more general and flexible graph learning paradigm in real-world scenarios.

large Language Models. In recent years, LLMs (*e.g.*, ChatGPT [29] and Claude [11]) have gained widespread attention for their remarkable capabilities in various NLP tasks [18, 46]. Based on these unique capabilities of LLMs, many tuning-free prompting techniques have been explored to enhance their generative abilities, such as in-context learning [28] and Chain-of-Thought [47, 57]. With the rise of open-source LLMs, such as Llama [36, 37], ChatGLM [62], and Baichuan [54], technologies for aligning pre-trained LLMs to different specific tasks and human feedback have been proposed, making private LLMs in specific domains possible [19, 44, 45].

While there have been successful attempts to align LLMs with visual information, such as multimodal LLMs [23, 66], the alignment of LLMs with graph structures remains largely unexplored.

This research addresses this gap by introducing a dual-stage graph instruction tuning paradigm that effectively aligns the language capacity of LLMs with graph learning. Previous studies [2, 5] have attempted to incorporate graph information into LLMs using natural language, but they have faced challenges in handling complex graph structures and achieving a deep understanding of graphs due to the limitations of relying solely on text-based prompts.

6 CONCLUSION

This work presents an effective and scalable graph large language model, aims at improving the generalization capabilities of graph models. The proposed framework, GraphGPT, injects graph domain-specific structural knowledge into the LLM through a dual-stage graph instruction tuning paradigm. By leveraging a simple yet effective graph-text alignment projector, we enable LLMs to comprehend and interpret the structural components of graphs. Extensive evaluations across different settings demonstrate the effectiveness of our method in both supervised and zero-shot graph learning scenarios. Furthermore, the model exhibits strong generalization abilities, allowing it to handle diverse downstream datasets and tasks without suffering from catastrophic forgetting. A potential avenue for future investigation is exploring pruning techniques to compress redundant or less important parameters of LLM, thereby reducing the overall model size while preserving its performance.

REFERENCES

- [1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. 2022. Constitutional AI: Harmlessness from AI Feedback. *CoRR* abs/2212.08073 (2022).
- [2] Zhikai Chen, Haitao Mao, Hang Li, et al. 2023. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *CoRR* abs/2307.03393 (2023).
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL-HLT (1)*. Association for Computational Linguistics, 4171–4186.
- [4] Yushun Dong, Ninghao Liu, Brian Jalaian, et al. 2022. EDITS: Modeling and Mitigating Data Bias for Graph Neural Networks. In *WWW. ACM*, 1259–1269.
- [5] Jiayan Guo, Lun Du, and Hengyu Liu. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data? An Empirical Evaluation and Benchmarking. *CoRR* abs/2305.15066 (2023).
- [6] Zhichun Guo, Kehan Guo, Bozhao Nan, Yijun Tian, Roshni G. Iyer, et al. 2023. Graph-based Molecular Representation Learning. In *IJCAI*. 6638–6646.
- [7] William L. Hamilton, Zitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NeurIPS*. 1024–1034.
- [8] Xiaoxin He, Xavier Bresson, et al. 2023. Explanations as Features: LLM-Based Features for Text-Attributed Graphs. *CoRR* abs/2305.19523 (2023).
- [9] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yong-Dong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *SIGIR. ACM*, 639–648.
- [10] Zhenyu Hou, Yufei He, Yukuo Cen, Xiao Liu, et al. 2023. GraphMAE2: A Decoding-Enhanced Masked Self-Supervised Graph Learner. In *WWW*. 737–746.
- [11] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Jie Tang, et al. 2022. Graphmae: Self-supervised masked graph autoencoders. In *KDD*. 594–604.
- [12] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, et al. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *NeurIPS*.
- [13] Ziniu Hu, Yuxiao Dong, Kuansan Wang, Kai-Wei Chang, and Yizhou Sun. 2020. Gpt-gnn: Generative pre-training of graph neural networks. In *KDD*. 1857–1867.
- [14] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous Graph Transformer. In *WWW. ACM / IW3C2*, 2704–2710.
- [15] Yangqin Jiang, Chao Huang, and Lianghao Huang. 2023. Adaptive graph contrastive learning for recommendation. In *KDD*. 4252–4261.
- [16] Baoyu Jing, Chanyoung Park, and Hanghang Tong. 2021. Hdmi: High-order deep multiplex infomax. In *WWW*. 2414–2424.
- [17] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR (Poster)*. OpenReview.net.
- [18] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. In *NeurIPS*.
- [19] Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, et al. 2023. RLAI: Scaling Reinforcement Learning from Human Feedback with AI Feedback. *CoRR* abs/2309.00267 (2023).
- [20] Bolian Li, Baoyu Jing, and Hanghang Tong. 2022. Graph communal contrastive learning. In *WWW*. 1203–1213.
- [21] Guohao Li, Matthias Müller, Bernhard Ghanem, and Vladlen Koltun. 2021. Training Graph Neural Networks with 1000 Layers. In *ICML*. 6437–6449.
- [22] Mingkai Lin, Wenzhong Li, Ding Li, Yizhou Chen, and Sanglu Lu. 2022. Resource-Efficient Training for Large Graph Convolutional Networks with Label-Centric Cumulative Sampling. In *WWW. ACM*, 1170–1180.
- [23] Haotian Liu, Chunyuan Li, et al. 2023. Visual Instruction Tuning.
- [24] Yixin Liu, Ming Jin, Shirui Pan, Chuan Zhou, Yu Zheng, Feng Xia, and S Yu Philip. 2022. Graph self-supervised learning: A survey. *TKDE* 35, 6 (2022), 5879–5900.
- [25] Yunchao Liu, Yu Wang, Oanh Vu, Rocco Moretti, et al. 2023. Interpretable Chirality-Aware Graph Neural Network for Quantitative Structure Activity Relationship Modeling in Drug Discovery. In *AAAI*. 14356–14364.
- [26] Zemin Liu, Xingtong Yu, et al. 2023. Graphprompt: Unifying pre-training and downstream tasks for graph neural networks. In *WWW*. 417–428.
- [27] Xiaojun Ma, Qin Chen, et al. 2022. Meta-Weight Graph Neural Network: Push the Limits Beyond Global Homophily. In *WWW. ACM*, 1270–1280.
- [28] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?. In *EMNLP*. 11048–11064.
- [29] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. 2022. Training language models to follow instructions with human feedback. In *NeurIPS*.
- [30] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning (ICML)*. PMLR, 8748–8763.
- [31] Zezhi Shao et al. 2022. Pre-training Enhanced Spatial-temporal Graph Neural Network for Multivariate Time Series Forecasting. In *KDD. ACM*, 1567–1577.
- [32] Kumar Shridhar, Alessandro Stolfo, and Mrinmaya Sachan. 2023. Distilling Reasoning Capabilities into Smaller Language Models. In *ACL*. 7059–7073.
- [33] Mingchen Sun, Kaixiong Zhou, et al. 2022. Gppt: Graph pre-training and prompt tuning to generalize graph neural networks. In *KDD*. 1717–1727.
- [34] Xiangguo Sun, Hong Cheng, Jia Li, Bo Liu, and Jihong Guan. 2023. All in One: Multi-Task Prompting for Graph Neural Networks. In *KDD*.
- [35] Qiaoyu Tan, Ninghao Liu, Xiao Huang, Soo-Hyun Choi, Li Li, Rui Chen, and Xia Hu. 2023. S2GAE: Self-Supervised Graph Autoencoders are Generalizable Learners with Graph Masking. In *WSDM*. 787–795.
- [36] Hugo Touvron, Thibaut Lavril, Gautier Lacroix, Baptiste Rozière, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023).
- [37] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023).
- [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, et al. 2017. Attention is all you need. In *NeurIPS*, Vol. 30.
- [39] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, et al. 2018. Graph Attention Networks. In *ICLR (Poster)*. OpenReview.net.
- [40] Petar Velickovic, William Fedus, William L. Hamilton, Pietro Liò, et al. 2019. Deep Graph Infomax. In *ICLR (Poster)*. OpenReview.net.
- [41] Kuansan Wang, Zhihong Shen, et al. 2020. Microsoft Academic Graph: When experts are not enough. *Quant. Sci. Stud.* 1, 1 (2020), 396–413.
- [42] Xiang Wang, Tinglin Huang, Dingxian Wang, et al. 2021. Learning Intents behind Interactions with Knowledge Graph for Recommendation. In *WWW*. 878–887.
- [43] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, et al. 2019. Heterogeneous Graph Attention Network. In *WWW. ACM*, 2022–2032.
- [44] Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, et al. 2023. How Far Can Camels Go? Exploring the State of Instruction Tuning on Open Resources. *CoRR* abs/2306.04751 (2023).
- [45] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *ACL*. 13484–13508.
- [46] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Jeff Dean, William Fedus, et al. 2022. Emergent Abilities of Large Language Models. *Trans. Mach. Learn. Res.* 2022 (2022).
- [47] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.
- [48] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2023. LLMRec: Large Language Models with Graph Augmentation for Recommendation. *CoRR* abs/2311.00423 (2023).
- [49] Zhihao Wen and Yuan Fang. 2023. Augmenting Low-Resource Text Classification with Graph-Grounded Pre-training and Prompting. In *SIGIR*.
- [50] Qitian Wu, Chenxiao Yang, et al. 2023. DIFformer: Scalable (Graph) Transformers Induced by Energy Constrained Diffusion. In *ICLR*.
- [51] Qitian Wu, Wentao Zhao, et al. 2023. NodeFormer: A Scalable Graph Structure Learning Transformer for Node Classification. *CoRR* abs/2306.08385 (2023).
- [52] Jun Xia, Lirong Wu, Jintao Chen, et al. 2022. Simgrace: A simple framework for graph contrastive learning without data augmentation. In *WWW*. 1070–1079.
- [53] Lianghao Xia, Chao Huang, Tao Yu, Ben Kao, et al. 2023. Automated Self-Supervised Learning for Recommendation. In *WWW*. 992–1002.
- [54] Aiyuan Yang, Bin Xiao, Bingning Wang, Borong Zhang, et al. 2023. Baichuan 2: Open Large-scale Language Models. *CoRR* abs/2309.10305 (2023).
- [55] Chenxiao Yang, Qitian Wu, and Junchi Yan. 2022. Geometric Knowledge Distillation: Topology Compression for Graph Neural Networks. In *NeurIPS*.
- [56] Haoran Yang, Hongxu Chen, Shirui Pan, Lin Li, Philip S Yu, and Guandong Xu. 2022. Dual space graph contrastive learning. In *WWW*. 1238–1247.
- [57] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *CoRR* abs/2305.10601 (2023).
- [58] Yuning You, Tianlong Chen, Yang Shen, and Zhangyang Wang. 2021. Graph contrastive learning automated. In *ICML. PMLR*, 12121–12132.
- [59] Yuning You, Tianlong Chen, Yongduo Sui, et al. 2020. Graph contrastive learning with augmentations. In *NeurIPS*, Vol. 33. 5812–5823.
- [60] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J. Kim. 2019. Graph Transformer Networks. In *NeurIPS*. 11960–11970.
- [61] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J. Kim. 2019. Graph transformer networks. In *NeurIPS*, Vol. 32.
- [62] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, et al. 2023. GLM-130B: An Open Bilingual Pre-trained Model. In *ICLR*.
- [63] Shichang Zhang, Yozen Liu, Yizhou Sun, and Neil Shah. 2022. Graph-less Neural Networks: Teaching Old MLPs New Tricks Via Distillation. In *ICLR*.
- [64] Wen Zhang, Yushan Zhu, Mingyang Chen, et al. 2023. Structure Pretraining and Prompt Tuning for Knowledge Graph Transfer. In *WWW*. 2581–2590.
- [65] Yanfu Zhang et al. 2022. Robust Self-Supervised Structural Graph Neural Network for Social Network Prediction. In *WWW. ACM*, 1352–1361.
- [66] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. 2023. MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models. *arXiv preprint arXiv:2304.10592* (2023).
- [67] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2021. Graph contrastive learning with adaptive augmentation. In *WWW*. 2069–2080.